Studio Nx

AI 구현 설명 82 — S1-11a 재생 오디오 그래프 (FR-201 · FR-205 · FR-241~252)

2026-10-19

AI 구현 문서 82 — S1-11a: 재생 오디오 그래프 (FR-201, FR-205, FR-241 ~ FR-252)

항목	내용
작업	S1-11a 재생 오디오 그래프: 오디오 클립 재생, 트랙 믹서, Synth, 이펙트 6종, 진짜 미터
작성	Claude (AI), 2026-10-19, 사용자 검토
기반	S1-10a 병합(develop), 결정 문서 Default_Instruments_Effects.md
프런트	`audio/` 폴더의 신규 파일 9개, `sequencer-audio.js`, 그리고 `sequencer-arrange.js`, `sequencer-devices.js`, `sequencer.js`의 연결 부분
백엔드	변경 없음 (시드에 Delay 한 줄 추가만, `studionx/db/02_phase2_sequencer.sql`)
함수 문서	JS 288 ~ 296 (9건), 그리고 별도 설명 문서 83 (Web Audio API)
시험	`docs/handoff/s1-11a_tests/`의 시험 9개, 합 115건 통과
이전 문서	81_s1-10a_connections_clips.md

---

1. 실행 결과

이 작업 전에는 시퀀서에서 소리가 전혀 나지 않았습니다. 재생 버튼을 누르면 재생선만 움직였고, 악기와 이펙트는 값을 저장하는 화면에 머물러 있었습니다. 이제 실제로 소리가 납니다.

확인한 장면	결과
오디오 클립을 놓고 ▶	사인파 소리가 나고, 마스터 출력 피크가 진폭과 일치합니다
트랙의 M(뮤트)	소리가 사라지고, 해제하면 돌아옵니다
다른 트랙의 S(솔로)	이 트랙은 무음이 됩니다
MIDI 클립의 노트	Synth가 A4(440Hz)를 연주합니다
트랙에 Gain -12dB	소리 크기가 약 1/4이 됩니다. 바이패스하면 원래 소리로 돌아옵니다
마스터에 Limiter	최종 출력의 피크가 줄어듭니다
미터	진폭 0.4의 소리에 약 87%를 보이고, 정지하면 천천히 0으로 내려갑니다

스크린샷과 동영상은 `docs/screenshots/2026-10-19_s1-11a_audio_graph/`에 있습니다.

파일	내용
`01_playing.png`	오디오 클립 재생 중
`02_midi_playing.png`	MIDI 노트 재생 중
`03_fx_base.png`, `03_fx_gain.png`, `03_fx_bypass.png`, `03_fx_gain24.png`, `03_fx_master_limiter.png`	이펙트 체인 시험 장면
`04_meter_playing.png`	미터가 소리 크기를 보여 주는 장면
`s1-11a_playback.mp4`	재생, 뮤트, 해제, 정지를 화면과 소리로 담은 동영상 (약 7.5초)

2. 무엇을 만들었나

2.1 신규 파일

파일	역할	함수 문서
`audio/mixer.js`	트랙별 볼륨, 팬, 뮤트, 솔로, 마스터, 이펙트 체인 자리를 만듭니다	288
`audio/playback.js`	오디오 클립을 언제, 샘플의 어디부터, 얼마나 울릴지 계산하고 재생합니다	289
`audio/session.js`	엔진, 믹서, 클립, 노트, 이펙트를 묶어 재생과 정지를 소리로 바꿉니다	290
`audio/synth.js`	감산 신스가 노트 하나를 소리로 냅니다	291
`audio/notes.js`, `audio/instruments.js`	MIDI 노트를 펼치고 트랙의 악기 층으로 연주합니다	292
`audio/effects.js`	Gain, EQ, Compressor, Limiter, Delay, Reverb를 Web Audio 노드로 만듭니다	293
`audio/effect_rack.js`	이펙트 목록을 체인으로 맞추고 노브 값을 반영합니다	294
`audio/meter.js`	소리 크기를 읽어 막대 높이로 바꿉니다	295
`pages/sequencer-audio.js`	화면의 값을 읽어 세션에 넘기고 재생 버튼을 소리에 연결합니다	296

2.2 수정한 파일

파일	바뀐 점
`sequencer-arrange.js`	재생 버튼과 소리를 잇는 훅(`setTransportHooks`) 추가
`sequencer-devices.js`	재생용 읽기 함수(`playbackTracks`, `playbackInstruments`, `playbackEffects`, `patternNotes`, `clipRecord`)와 진짜 미터 추가
`sequencer.js`	`initAudio()` 호출 추가
`studionx/db/02_phase2_sequencer.sql`	시스템 플러그인 시드에 Delay 추가 (기존 DB용 `migrations/2026-10-19_plugin_delay.sql`)

2.3 소리가 흐르는 길

```
오디오 클립 ─▶ 클립 게인 ─┐
                          ├─▶ [트랙 source] ─▶ 이펙트 체인 ─▶ [볼륨, 뮤트, 솔로] ─▶ [팬] ─┐
MIDI 노트 ─▶ Synth ─▶ 악기 층(볼륨, 팬) ─┘                                                  │
                                                                                            ▼
                                  (다른 트랙들) ──────────────────────────────────────▶ [마스터 볼륨] ─▶ 마스터 이펙트 ─▶ [out] ─▶ 스피커

                                  │

                                  미터, 시험의 피크 측정이 여기서 읽음
```

3. 요구사항 대응

요구사항	내용	상태
FR-201	재생, 일시정지, 정지가 실제 소리로 동작합니다	반영 (재생 중 재생선 이동은 소리에 반영되지 않음)
FR-205	클립과 노트가 정확한 시각에 울립니다 (lookahead 예약)	반영
FR-241, FR-242	트랙의 악기가 노트를 연주합니다	Synth만 반영, Sampler는 MIDI로 소리가 나지 않음
FR-243, FR-247	이펙트 6종이 Web Audio 기본 노드로 동작합니다	반영 (Limiter는 근사치)
FR-251	트랙 볼륨, 팬, 뮤트, 솔로, 미터	반영
FR-252	마스터 트랙과 마스터 이펙트	반영
FR-263	소스, 이펙트 체인, 트랙 출력의 직렬 체인	반영 (병렬 연결은 S2-02 패처)

4. 왜 이렇게 (이번 작업의 결정)

결정	이유
작업을 S1-11a(재생 그래프)와 S1-11b(WAV 출력)로 나눴습니다	확인해 보니 시퀀서에 소리가 나는 경로가 아예 없었습니다. WAV 출력은 "들리는 소리와 같아야" 하므로 재생 그래프가 먼저입니다. 계획서의 일정을 2주 늘렸습니다.
같은 코드가 실시간과 WAV 출력에 쓰이게 만들었습니다	모든 오디오 코드가 컨텍스트를 인자로 받습니다. `AudioContext`와 `OfflineAudioContext`를 바꿔 끼우기만 하면 됩니다. 이펙트 시험이 이미 이 방식으로 동작합니다.
Web Audio 기본 노드로 구현했습니다	설치가 필요 없고 JavaScript만으로 가능합니다. 정밀한 DSP는 이후 사용자의 C++(WASM)로 교체합니다 (`engine='WASM'`). 설명은 문서 83에 있습니다.
Delay를 1단계에 추가했습니다	Ableton Live와 FL Studio를 비교하니 두 DAW 모두 딜레이를 기본으로 제공했고, 시드에는 없었습니다 (결정 문서).
Sampler는 MIDI로 연주하지 않게 했습니다 (B안)	시드의 Sampler에는 샘플을 고르는 칸이 없습니다. 목업을 본 뒤 One-Shot 모드와 함께 결정하기로 했습니다.
시험을 실제 소리 측정으로 했습니다	소리 코드는 눈으로 확인할 수 없어서, `AnalyserNode`로 마스터 출력의 피크를 재고 `OfflineAudioContext`로 이펙트 결과를 계산해서 숫자로 확인합니다.
노드 연결 구조를 "소스, 이펙트, 볼륨"으로 했습니다	실제 DAW처럼 이펙트가 페이더보다 앞에 있어야 볼륨을 내려도 이펙트의 동작이 같습니다.

5. 시험 중에 발견해서 고친 문제

시험이 실패해서 발견한 것을 모두 기록해 둡니다. 코드를 짠 직후 실제 소리를 재지 않았다면 놓쳤을 문제들입니다.

문제	원인	해결
모노 소리가 가운데에서 -3dB가 됩니다	`StereoPannerNode`의 팬 법칙이 모노 입력에 0.707배를 적용합니다	입구를 스테레오로 고정했습니다 (`createStereoGain`)
Limiter가 천장을 넘는 소리를 냅니다	`DynamicsCompressorNode`가 자동으로 메이크업 게인을 더합니다	계산해서 같은 양을 줄였습니다 (`compressorAutoMakeupDb`)
딜레이의 반복이 박자에서 늦어집니다	되먹임 고리를 돌 때마다 브라우저가 128샘플을 더 걸리게 합니다	고리의 딜레이에서 128샘플을 빼고 첫 반복을 따로 늦췄습니다
미터 막대가 믹서 줄에서 움직이지 않습니다	`startMeters`가 펼침 칸을 열 때만 불렸습니다	재생을 시작할 때 타이머를 켜고, 모두 0이면 끕니다

문제	원인	해결
MIDI 시험에서 소리가 거의 0입니다	그 트랙의 기존 Synth에 컷오프 20Hz가 저장되어 있었고 볼륨이 -21.5dB였습니다 (코드 문제가 아님)	시험이 기본값의 새 Synth를 붙이고 볼륨을 0으로 맞춥니다
오디오 시험에서 소리가 0으로 측정됩니다	예시 화면의 재생선이 12.5마디에서 시작했고, 1초 샘플은 측정 전에 끝났습니다 (시험 설계 문제)	■로 재생선을 0으로 되돌리고 8초 샘플을 씁니다
이펙트 시험의 기준 소리가 달랐습니다	기존 오디오 트랙에 이펙트가 이미 있었습니다	이펙트가 없는 시험 전용 트랙을 새로 만들어 씁니다

6. 검증 결과

확인	방법	결과
믹서와 클립 재생 순수 함수	`mixer_playback_unit.mjs`	12건 통과
Synth와 노트 순수 함수	`synth_notes_unit.mjs`	20건 통과
이펙트 순수 함수	`effects_unit.mjs`	17건 통과
미터 순수 함수	`meter_unit.mjs`	14건 통과
이펙트 6종의 실제 소리 (`OfflineAudioContext`)	`effects_offline.mjs`	16건 통과
오디오 클립, 뮤트, 솔로, 정지 (실제 화면)	`ui.mjs`	10건 통과
MIDI 노트의 Synth 소리 (실제 화면)	`ui_midi.mjs`	8건 통과
이펙트 체인, 바이패스, 마스터 Limiter (실제 화면)	`ui_fx.mjs`	8건 통과
진짜 미터 (실제 화면)	`ui_meter.mjs`	10건 통과
백엔드	`python -m pytest -q`	1083건 통과
시험 데이터 정리	시험 뒤 DB 조회	클립, 패턴, 샘플, 시험 트랙이 남지 않았고 기존 트랙 값이 그대로입니다

시험 실행 방법은 파일 맨 위 주석에 적혀 있고, 모두 `node docs/handoff/s1-11a_tests/<이름>.mjs` 입니다. 화면 시험은 시험용 서버(8001)와 시험 계정이 필요합니다.

7. 알려진 한계

한계	설명
Sampler는 MIDI 노트로 소리가 나지 않습니다	샘플을 고르는 칸이 없습니다 (결정 보류)
재생 중 재생선을 옮겨도 소리는 따라가지 않습니다	정지 후 다시 재생해야 합니다
곡 중간 템포 변경은 소리에 반영되지 않습니다	상단 BPM 값만 씁니다
노트 중간에서 시작하면 그 노트는 울리지 않습니다	재생 위치 이후에 시작하는 노트만 울립니다
노트의 확률, 속도 편차, CC, 피치벤드는 반영하지 않습니다	이후 작업입니다

한계	설명
클립의 페이드, 거꾸로 재생, 워프는 반영하지 않습니다	게인, 루프, 음정(반음)은 반영합니다
악기 층의 Solo 버튼은 반영하지 않습니다	트랙 솔로만 반영합니다
재생 중 파라미터 변경은 이미 울리는 노트에 반영되지 않습니다	다음 노트부터 적용됩니다
Limiter는 근사치입니다	정확한 피크 리미터가 아닙니다 (이후 WASM)
이펙트를 바이패스했다가 켜면 딜레이와 리버브의 꼬리가 이어지지 않습니다	켜는 순간 새로 만들어집니다
볼륨과 팬 노브 변경은 약 0.15초 간격으로 반영됩니다	믹서 갱신 타이머 간격입니다
사람의 귀로 듣는 확인은 일부만 받았습니다	사용자가 귀로 확인한 것은 오디오 클립 재생(1 ~ 3단계)까지입니다. Synth, 이펙트, 미터는 헤드리스 시험(피크와 계산된 소리 크기)으로만 확인했고, 사용자 검토에서 들어 보셔야 합니다

8. 남은 일과 다음 작업이 쓰는 곳

작업	쓰는 것
S1-11b WAV 출력	`OfflineAudioContext`에 `MixerGraph`, `InstrumentRack`, `EffectRack`, `ClipPlayer`를 만들어 같은 그래프를 렌더링합니다. `PlaybackSession`의 시작 로직을 오프라인용으로 분리하면 됩니다
S1-12 회귀 테스트	이펙트 시험의 렌더링 방식을 그대로 확장해 골든 WAV와 비교합니다. 리버브 임펄스를 고정 시드로 만든 것이 재현성의 기반입니다
S2-03	마스터 Limiter와 Gain의 설정 화면, 프리셋 저장과 불러오기 (Delay는 이미 만들었음)
Sampler 결정	화면 목업을 확인한 뒤 One-Shot 모드와 샘플 선택 칸을 정합니다

9. 자주 받을 질문 대비 (Q/A)

Q. 소리는 어떻게 내나요? A. 브라우저의 Web Audio API로 노드를 만들어 연결합니다. 자세한 설명은 문서 83에 있습니다.

Q. 왜 JavaScript로 가능한가요? A. 소리 계산은 브라우저의 네이티브 엔진이 하고, JavaScript는 노드 연결 지시만 하기 때문입니다. 정밀한 처리가 필요하면 나중에 C++(WASM)로 바꿉니다.

Q. 실시간 재생과 WAV 출력의 소리가 같은가요? A. 같은 코드가 컨텍스트만 바꿔 쓰이도록 만들었기 때문에 같습니다. 이펙트 시험은 이미 `OfflineAudioContext`로 렌더링해서 확인합니다. 잔향 임펄스를 고정 시드로 만들어서 같은 설정이면 결과도 같습니다.

Q. 이펙트가 정말 소리를 바꾼다는 증거는 무엇인가요? A. `effects_offline.mjs`가 이펙트마다 입력과 출력의 크기를 숫자로 비교합니다. 예를 들어 Gain -12dB는 크기가 0.251배이고, EQ 저음 -24dB는 100Hz 소리를 30% 아래로 줄입니다. 화면 시험(`ui_fx.mjs`)도 바이패스 전후의 피크를 비교합니다.

Q. 미터가 진짜라는 증거는 무엇인가요? A. 진폭 0.4의 사인파는 -8dB이므로 약 87%여야 합니다. 시험이 실제로 80에서 93% 사이이고 흔들리지 않음을 확인합니다. 흉내 값(무작위)은 이 수치와 맞지 않습니다.

Q. 소리가 안 날 때 먼저 어디를 보나요? A. 순서대로 봅니다. 트랙 볼륨, 뮤트, 솔로, 악기의 컷오프 같은 극단 값, 이펙트의 값을 확인합니다. 이 작업 중에 컷오프 20Hz와 볼륨 -21.5dB 때문에 소리가 거의 안 나는 경우가 실제로 있었습니다.

Q. Sampler는 왜 MIDI로 소리가 안 나나요? A. 어떤 샘플로 노트를 연주할지 고르는 칸이 시드에 없습니다. 오디오 클립용으로는 정상 동작합니다. 목업을 확인한 뒤 결정합니다.

Q. Delay는 왜 추가했나요? A. Ableton Live와 FL Studio를 비교했을 때 두 DAW 모두 딜레이를 기본으로 제공했고, 시드에는 없었기 때문입니다. 근거는 Default_Instruments_Effects.md에 있습니다.

Q. 동시에 몇 개의 노트까지 연주되나요? A. 제한을 두지 않았습니다. 노트마다 오실레이터를 새로 만들어서 매우 많으면 무거워질 수 있고, 이후 제한을 둘 수 있습니다.